

Lab 10: Spring WebFlux & WebClient

1. Reactive Programming vs Blocking

Reactive Programming คือการเขียนโปรแกรมแบบ asynchronous ที่ทำงานกับข้อมูลในรูปแบบ data stream โดยใช้แนวคิด Publisher → Operator → Subscriber เมื่อข้อมูลพร้อม Publisher จะส่งสัญญาณ onNext, onError หรือ onComplete ไปยัง Subscriber เอง ทำให้ thread ไม่ต้องหยุดรอผลลัพธ์

ในแบบ Blocking (เช่น Spring MVC บน Tomcat) แต่ละ request จะใช้ thread หนึ่งตัว และ thread นั้นจะหยุดรอนานกว่างาน I/O เช่น การอ่านฐานข้อมูลหรือการเรียก API ภายนอกจะเสร็จ ถ้ามี request พร้อมกันจำนวนมาก thread จะหมด ส่วนแบบ Reactive (Spring WebFlux บน Reactor Netty) ใช้ thread จำนวนน้อยแบบ event loop เมื่อต้องรอ I/O thread จะปล่อยงานนั้นไว้แล้วไปรับ request อื่นทันที เมื่อข้อมูลมาถึงจึงค่อยทำงานต่อ

หัวข้อ	Blocking (แบบเดิม)	Reactive (WebFlux)
การทำงานของ thread	รอนานกว่าจะได้ผล	ไม่รอ ไปทำงานอื่นได้ทันที
จำนวน thread	1 thread ต่อ 1 request	thread น้อย รองรับ request มาก
ชนิดค่าที่คืน	Product, List<Product>	Mono<Product>, Flux<Product>
Server	Tomcat (Servlet)	Reactor Netty (non-blocking)
เหมาะกับ	CRUD ทั่วไป, โหลดไม่สูง	งาน concurrency สูง, streaming

ข้อสำคัญ: Publisher เป็นเพียงแผนการทำงาน (blueprint) จะยังไม่มีอะไรเกิดขึ้นจนกว่าจะมีการเรียก subscribe() ใน Controller ของ WebFlux ตัว framework จะเป็นผู้ subscribe ให้เองเมื่อต้องส่ง response

2. Mono vs Flux

Mono<T> คือ Publisher ที่ส่งข้อมูลได้ 0 หรือ 1 ค่า ใช้เมื่อผลลัพธ์มีได้แค่ค่าเดียว เช่น findById, save, หรือราคาหลังส่วนลดของสินค้าหนึ่งชิ้น ถ้าไม่พบข้อมูลจะได้ Mono.empty() ส่วนงานที่ไม่มีข้อมูลคืน เช่น delete จะใช้ Mono<Void> ซึ่งส่งแค่สัญญาณว่าเสร็จแล้ว

Flux<T> คือ Publisher ที่ส่งข้อมูลได้ 0 ถึง N ค่า ใช้เมื่อผลลัพธ์มีหลายรายการ เช่น findAll, การค้นหาตาม category หรือ event stream ข้อมูลแต่ละตัวจะถูกส่งออกไปทีละตัวเมื่อพร้อม

เหตุผลที่ต้องแยกเป็นสองชนิด คือช่วยให้ชนิดข้อมูลบอกความหมายได้ชัดเจนว่าจะได้ผลลัพธ์กี่ค่า และเลือก operator ได้เหมาะสม เช่น Mono ใช้ defaultIfEmpty หรือ switchIfEmpty จัดการกรณีไม่พบข้อมูล ส่วน Flux ใช้ filter, take หรือ collectList จัดการกับหลายรายการ

Endpoint ในแลปนี้	ชนิดที่คืน	เหตุผล
GET /products/{id}	Mono<Product>	สินค้า 1 ชิ้น
GET /products	Flux<Product>	สินค้าทุกชิ้น
POST /products	Mono<Product>	สินค้าที่บันทึก 1 ชิ้น
DELETE /products/{id}	Mono<Void>	ไม่มีข้อมูลคืน
GET /products/category/{cat}	Flux<Product>	หลายชิ้นที่ผ่าน filter
GET /products/{id}/price	Mono<Double>	ราคาเดียวจาก map

Operators ที่ใช้ในแลป

- **map** แปลงค่าแบบ synchronous เช่น Product → ราคาหลังส่วนลด (Service) หรือ Product → ชื่อ (WebClient)
- **filter** กรองเฉพาะสินค้าที่ category ตรงกัน (Repository)
- **switchIfEmpty** เปลี่ยน Mono ว่างให้เป็น error เมื่อไม่พบสินค้า (Service)
- **defaultIfEmpty** ค่าสำรองเมื่อ stream ว่าง (WebClient demo)
- **doOnNext** ทำ side-effect เช่น log ราคาโดยไม่เปลี่ยนค่า (WebClient)
- **subscribe** เริ่มการทำงานของ stream

3. WebClient และ method chain

WebClient คือ HTTP client แบบ non-blocking ของ Spring WebFlux ใช้แทน RestTemplate ผลลัพธ์ที่ได้เป็น Mono หรือ Flux จึงนำไป chain operator ต่อได้ทันที โดยไม่ต้องหยุดรอ response

```
Mono<Product> p = client.get()           // 1. เลือก HTTP method
    .uri("/products/{id}", "1")          // 2. กำหนด URL + path variable
```

```
.retrieve() // 3. ส่ง request และเตรียมอ่าน response
.bodyToMono(Product.class); // 4. แปลง body เป็น Mono<Product>
```

```
Flux<Product> all = client.get()
    .uri("/products")
    .retrieve()
    .bodyToFlux(Product.class); // body เป็น array -> Flux<Product>
```

- **.get() / .post() / .delete()** เลือก HTTP method ที่จะส่ง
- **.uri(...)** กำหนด path ของ endpoint โดยใส่ตัวแปรแทน {id} ได้ ต่อจาก base URL ที่กำหนดตอนสร้าง (http://localhost:8080)
- **.bodyValue(obj)** ใช้กับ POST เพื่อส่ง object เป็น JSON body
- **.retrieve()** ส่ง request และเตรียมอ่าน response ถ้า status เป็น 4xx/5xx จะกลายเป็นสัญญาณ onError
- **.bodyToMono(T.class)** แปลง body เป็น Mono<T> ใช้เมื่อ response เป็นค่าเดียว
- **.bodyToFlux(T.class)** แปลง body ที่เป็น array เป็น Flux<T> ใช้เมื่อ response มีหลายรายการ

ในแลปนี้เรียกใช้ WebClient ผ่าน CommandLineRunner ตอนแอปเริ่มทำงาน โดย chain

```
.map().defaultIfEmpty().subscribe() ต่อจากผลลัพธ์ทันที (ผลลัพธ์ดูได้ที่หัวข้อ 5.2)
```

// ใน Lab10Application.java — เรียก WebClient ตอนแอปเริ่มทำงาน

```
@Bean
```

```
CommandLineRunner demo(ProductWebClient webClient) {
```

```
    return args -> {
```

```
        webClient.getAllProducts()
```

```
            .map(p -> p.getName()) // แปลง Product -> ชื่อ
```

```
            .defaultIfEmpty("No products") // fallback ถ้าว่าง
```

```
            .subscribe(name -> // เริ่มทำงาน (non-blocking)
```

```
                System.out.println("[WebClient] " + name));
```

```
        webClient.getDiscountedPrice("1").subscribe();
```

```
    };
```

```
}
```

4. Code ของ Repository, Service, Controller และ WebClient

4.1 ProductRepository.java

ใช้ Mono.justOrEmpty สำหรับค้นหาสินค้า 1 รายการ, Flux.fromIterable สำหรับดึงทั้งหมด และใช้ operator filter กรองสินค้าตาม category

```
// — 1. หา Product 1 รายการ —————

public Mono<Product> findById(String id) {
    // TODO: เติม code ตรงนี้
    return Mono.justOrEmpty(store.get(id));
}

// — 2. หา Product ทั้งหมด —————

public Flux<Product> findAll() {
    // TODO: เติม code ตรงนี้
    return Flux.fromIterable(store.values());
}

// — 3. บันทึก Product —————

public Mono<Product> save(Product product) {
    // TODO: เติม code ตรงนี้
    store.put(product.getId(), product);
    return Mono.just(product);
}

// — 4. ลบ Product —————

public Mono<Void> deleteById(String id) {
    // TODO: เติม code ตรงนี้
    store.remove(id);
    return Mono.empty();
}

// — 5. กรองตาม category —————

public Flux<Product> findByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return findAll().filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

รูปที่ 1 Code ของ ProductRepository

4.2 ProductService.java

ใช้ switchIfEmpty เปลี่ยน Mono ว่างให้เป็น error เมื่อไม่พบสินค้า สร้าง UUID ให้สินค้าใหม่ที่ไม่มี id และใช้ operator map แปลง Product เป็นราคาหลังส่วนลด

```
// — 1. ดึง Product 1 รายการ —————
public Mono<Product> getById(String id) {
    return repository.findById(id).switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
}

// — 2. ดึง Product ทั้งหมด —————
public Flux<Product> getAll() {
    return repository.findAll();
}

// — 3. บันทึก Product —————
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

// — 4. ลบ Product —————
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

// — 5. กรองตาม category —————
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// — 6. คำนวณราคาหลังส่วนลด —————
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id).map(Product::getDiscountedPrice);
}
```

รูปที่ 2 Code ของ ProductService

4.3 ProductController.java

แต่ละ endpoint คืนค่า Mono หรือ Flux จาก Service ทันทีโดยไม่รอผล แล้ว WebFlux จะ subscribe และส่ง response เมื่อข้อมูลพร้อม

```
// =====
// ✖ TODO: เติม method body ด้านล่างนี้
// =====

@GetMapping
public Flux<Product> getAll() {
    // TODO: เติม code ตรงนี้
    return service.getAll();
}

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    // TODO: เติม code ตรงนี้
    return service.save(product);
}

@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.delete(id);
}

@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    // TODO: เติม code ตรงนี้
    return service.getByCategory(category);
}

@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    // TODO: เติม code ตรงนี้
    return service.getDiscountedPrice(id);
}
```

รูปที่ 3 Code ของ ProductController

4.4 ProductWebClient.java

ใช้ WebClient เรียก endpoint ผ่าน method chain .get().post().delete() → .uri() → .retrieve() → .bodyToMono().bodyToFlux() และใช้ doOnNext เพื่อ log ราคา

```
// =====
// ✖ TODO: เติม method body ด้านล่างนี้
// =====

public Flux<Product> getAllProducts() {
    // TODO: เติม code ตรงนี้
    return client.get().uri("/products").retrieve().bodyToFlux(Product.class);
}

public Mono<Product> createProduct(Product product) {
    // TODO: เติม code ตรงนี้
    return client.post().uri("/products").bodyValue(product).retrieve().bodyToMono(Product.class);
}

public Mono<Void> deleteProduct(String id) {
    // TODO: เติม code ตรงนี้
    return client.delete().uri("/products/{id}", id).retrieve().bodyToMono(Void.class);
}

public Flux<Product> getByCategory(String category) {
    // TODO: เติม code ตรงนี้
    return client.get().uri("/products/category/{category}", category).retrieve().bodyToFlux(Product.class);
}

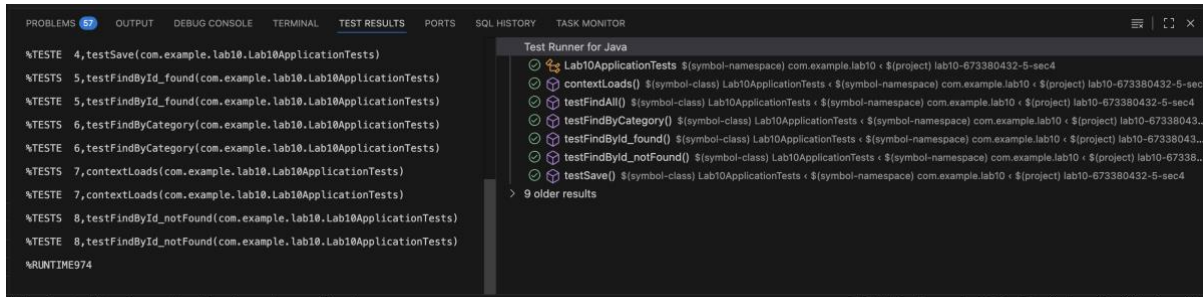
public Mono<Double> getDiscountedPrice(String id) {
    // TODO: เติม code ตรงนี้
    return client.get().uri("/products/{id}/price", id).retrieve().bodyToMono(Double.class)
        .doOnNext(price -> System.out.println("Price: " + price));
}
```

รูปที่ 4 Code ของ ProductWebClient

5. ผลการทดสอบ

5.1 Unit Test ด้วย StepVerifier

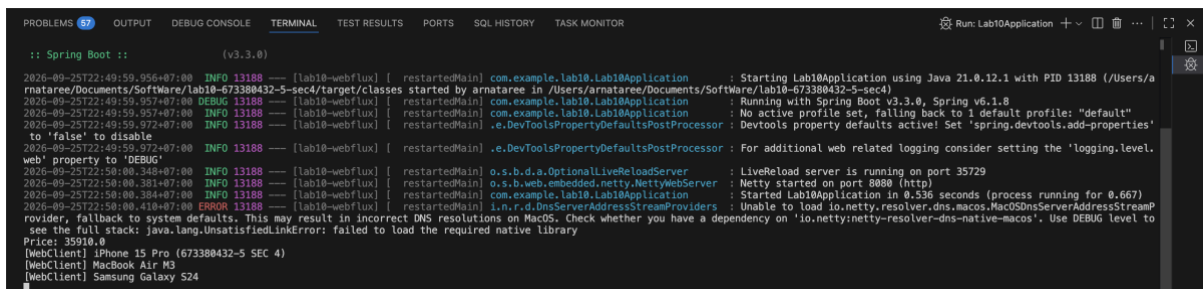
ทดสอบ Repository ด้วย StepVerifier ครบ 7 test ได้แก่ contextLoads, findById (พบ/ไม่พบ), findAll (3 รายการ), save และ findByCategory("Electronics") ผ่านทั้งหมด



รูปที่ 5 ผลการรัน Unit Test ผ่านทั้ง 7 test

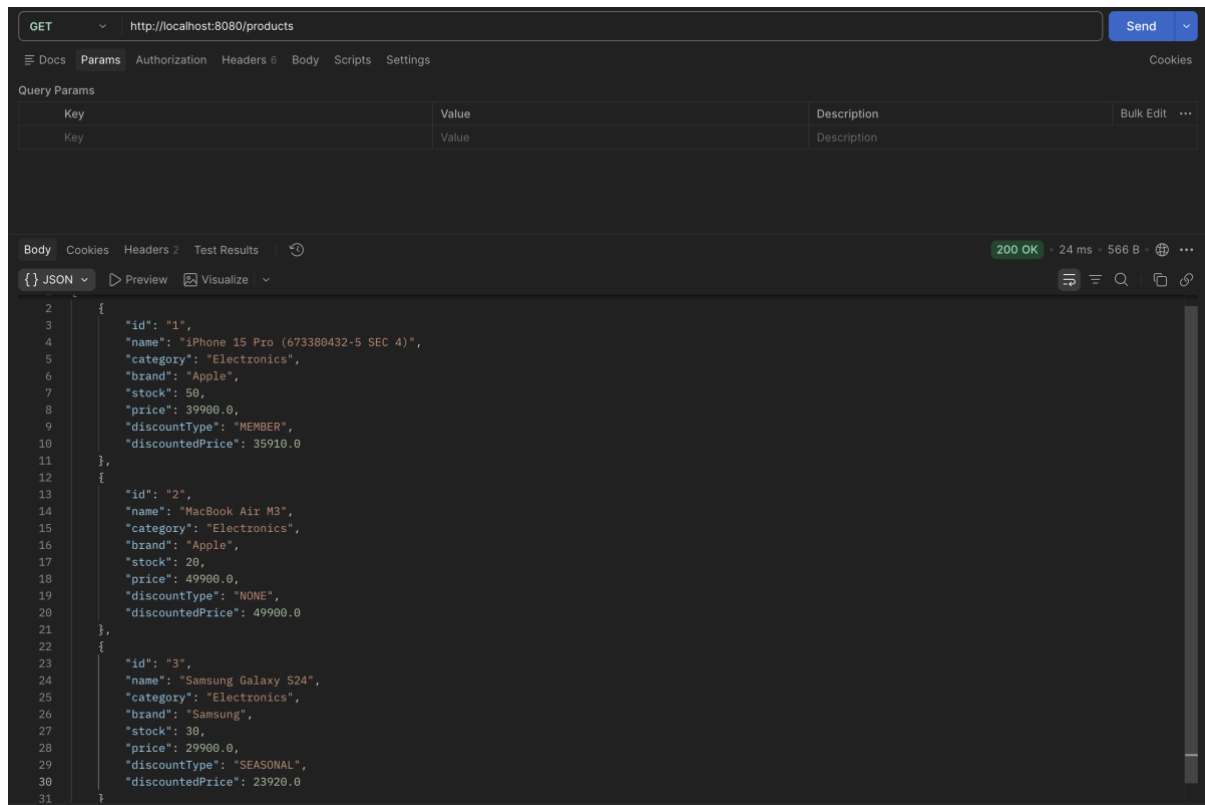
5.2 ผลลัพธ์จาก WebClient

เมื่อรันแอป WebClient เรียก GET /products แล้ว map เป็นชื่อสินค้าและพิมพ์ออกมา 3 รายการ และเรียก /products/1/price ได้ราคาหลังส่วนลด 35910.0

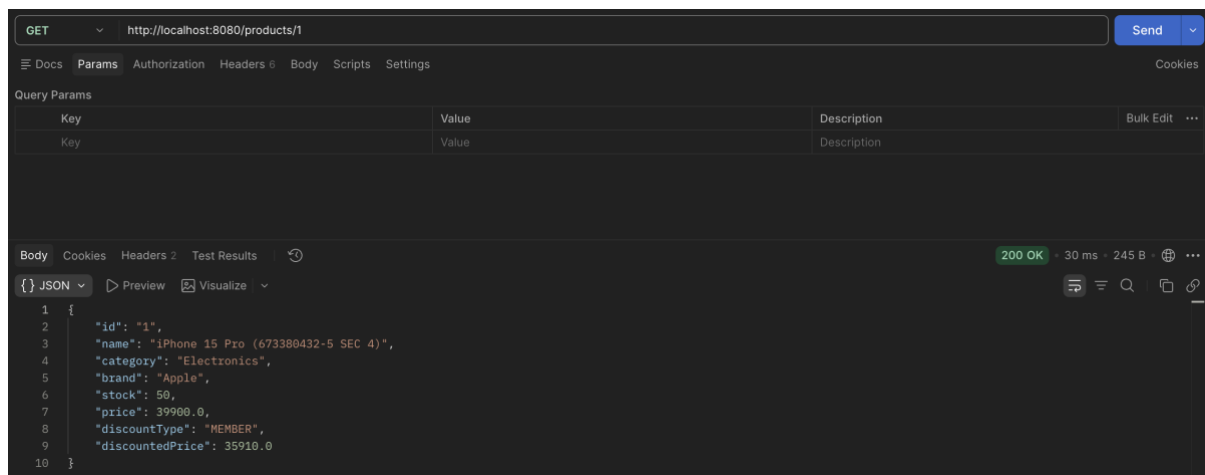


รูปที่ 6 Log ของ WebClient ใน Terminal

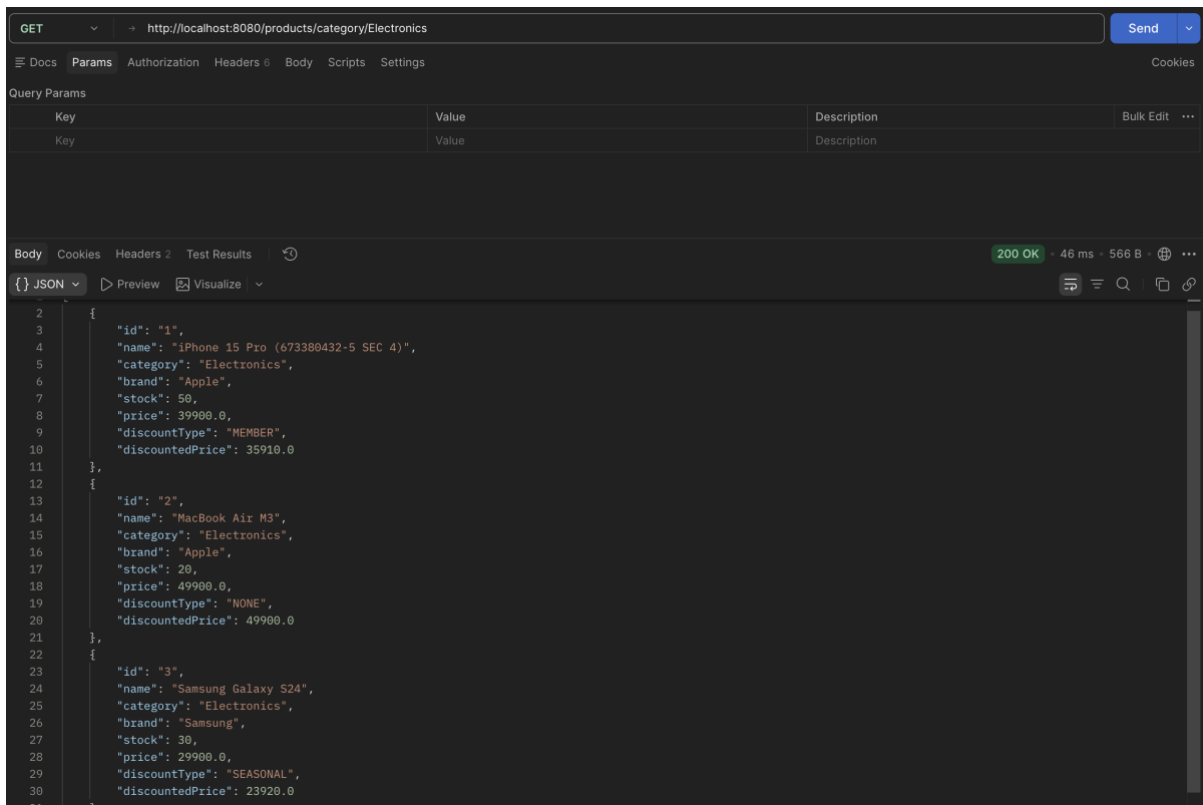
5.3 ทดสอบ Endpoint ด้วย Postman



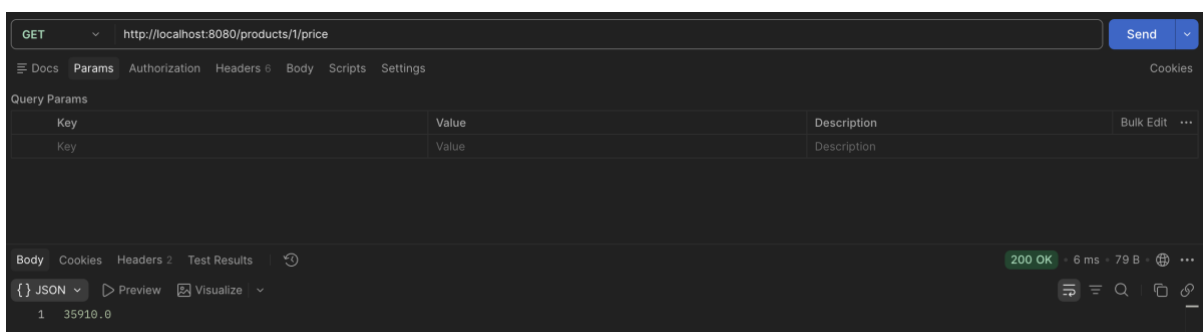
รูปที่ 7 GET /products — คืน Flux<Product> 3 รายการ



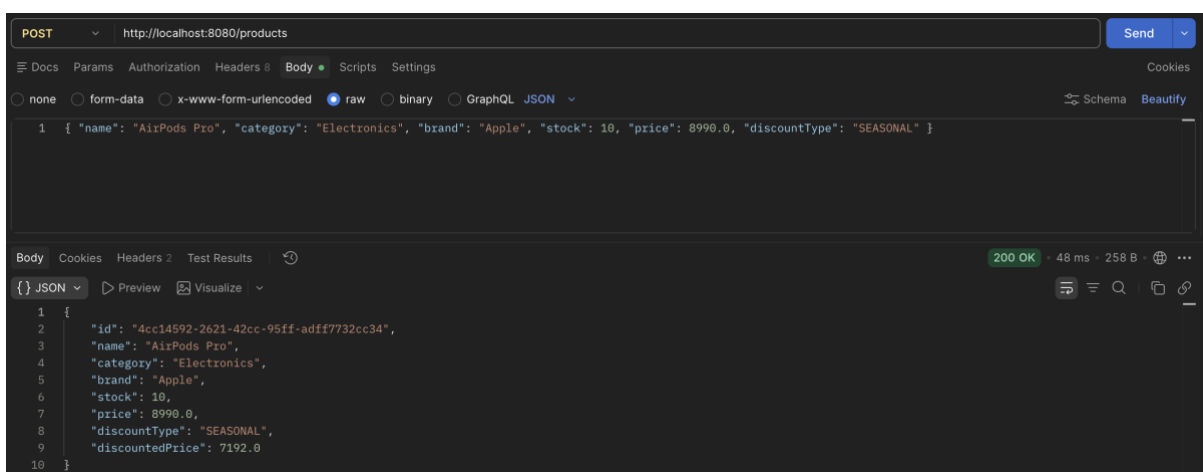
รูปที่ 8 GET /products/1 — คืน Mono<Product>



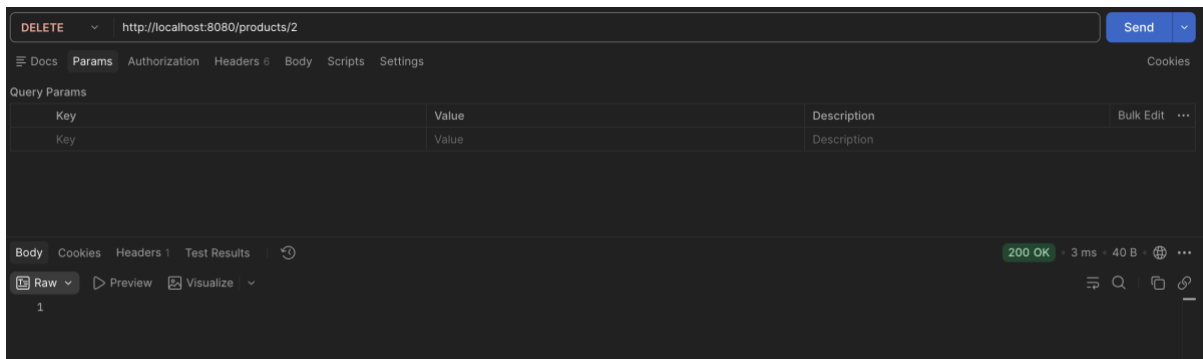
รูปที่ 9 GET /products/category/Electronics — กรองด้วย filter



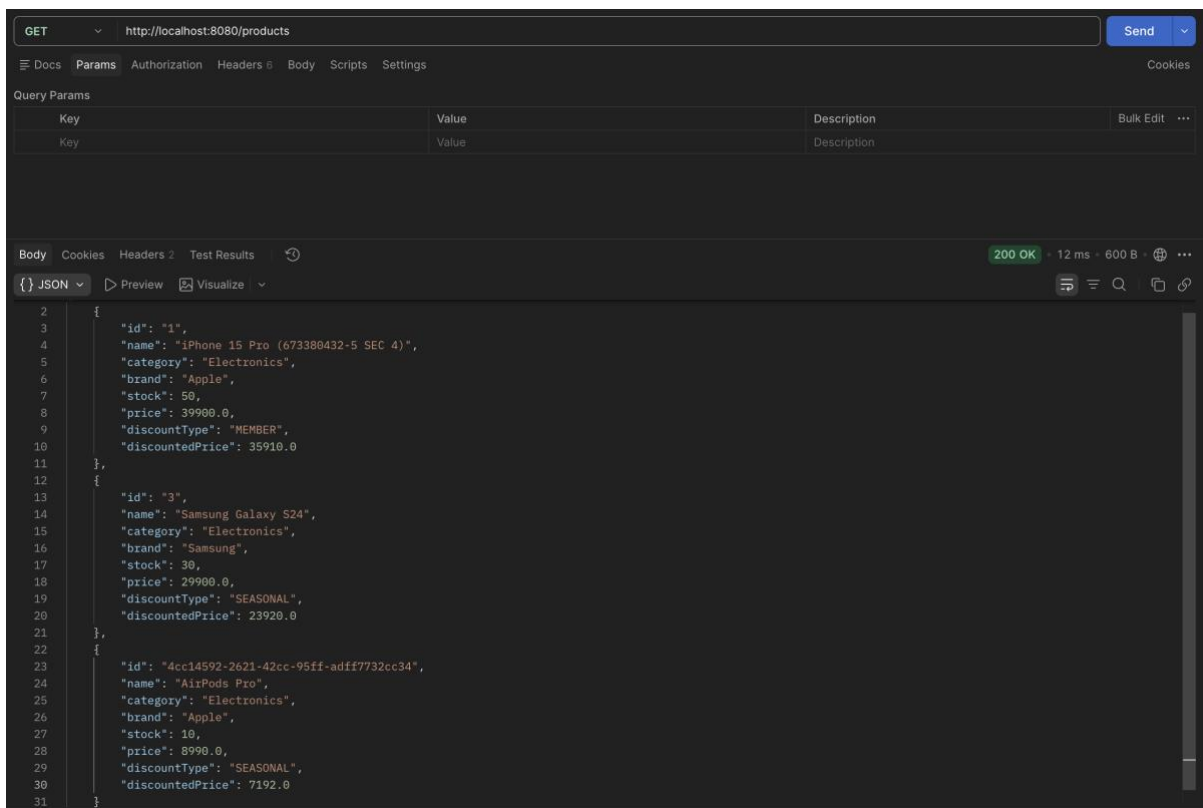
รูปที่ 10 GET /products/1/price — ราคาหลังส่วนลด MEMBER 10% = 35910.0



รูปที่ 11 POST /products — สร้างสินค้าใหม่ ได้ id เป็น UUID



รูปที่ 12 DELETE /products/2 — 200 OK และ body ว่าง (Mono<Void>)



รูปที่ 13 GET /products หลัง POST และ DELETE — MacBook หายไป มี AirPods Pro เพิ่ม